Emergency Training Simulator POC

Submission Packet

Roadmap

Project roadmap

Source: docs/roadmap.md

Prepared for review, discussion, and project handoff.

Markdown stays editable. This PDF is the clean reading copy.

Boundary: this is still a POC, not operational or clinical guidance.

Roadmap

This roadmap separates the current POC path from the full-system path.

That separation matters.

The full project vision is large. The current repo should grow toward it carefully, one trusted layer at a time.

For a direct comparison between the original full-system goals, the current repo, the remaining gaps, and the safest next experiments, see full-system-gap-map.md.

Solo-developer framing

This is currently a solo-developer project.

That means the roadmap should not be read as "build the whole final simulator next."

It should be read as:

```
large full-system vision
-> small proof
-> working checkpoint
-> reviewed result
-> next thin slice
```

The goal is steady progress without letting Unreal, generated assets, AI, sensing, physics, or deployment swallow the project before the foundation is ready.

Track 1: Current POC milestones

These are the milestones for this repository right now.

Completed

- Sprint 0 repo scaffolding
- deterministic backend scenario loop
- FastAPI WebSocket session endpoint
- JSON message contracts
- local JSONL session logs
- replay and review scripts
- one-command local demo
- wooden chair environment cue
- entry door environment cue
- future Unreal client contract
- text-only Unreal display preview
- JSON-to-debug-text bridge for the first Unreal panel
- first security baseline
- local demo abuse guardrails
- lightweight GitHub Actions CI
- UX docs for Dr. Cameron as first audience
- Dr. Cameron review rubric and scope-control rubric
- full-system gap map
- second deterministic toy scenario
- improved post-scenario review format
- project presentation package and PDF

Near-term

- keep improving the review/debrief output
- add richer instructor/reviewer examples
- make Unreal read the generated debug text file automatically
- parse the display JSON directly after the text path works
- add clearer message-contract versioning notes
- keep scenario logic deterministic and backend-owned

Later POC

- add more environment objects with obvious state
- add richer replay comparison reports
- add configurable local scenarios
- add thin Unreal WebSocket experiment only after file/JSON display is stable

Track 2: Full-system milestones

These belong to the larger project vision.

They should not be treated as current repo features yet.

Future AI layer

- AI-assisted scenario generation
- agentic scenario adaptation
- model-based trainee action interpretation
- AI-supported review summaries
- validation of AI-generated outputs

Future validation layer

- structured medical/protocol references
- department-provided validated materials
- rule-based and model-based validation
- update process for changing guidelines
- review workflow for validation data

Future immersive environment layer

- Unreal scene development
- projection mapping experiments
- mannequin localization
- projected patient/environment changes
- synchronized audio/visual feedback

Future sensing layer

- voice input
- pose and gesture recognition
- motion tracking
- procedural action detection
- sensor confidence and uncertainty handling

Future deployment layer

- cloud services
- hub-and-spoke architecture
- Virginia Tech hub role

- spoke-site deployment model
- resilience and caching
- multi-site instructor support

Recommended build order

The safest build order is:

1. trusted backend loop
2. replay and review
3. message contracts
4. security and trust boundaries
5. UX and instructor review flow
6. future client display contract
7. richer deterministic scenarios
8. Unreal text/debug client
9. generated/reviewed asset pipeline
10. real physics experiments
11. sensing experiments
12. AI adaptation experiments
13. validation framework expansion
14. deployment architecture

Rough timeline

This is a planning timeline, not a promise.

The timing depends on solo-developer bandwidth, Unreal setup friction, compute access, Dr. Cameron's feedback, and how small the next slices stay.

Current checkpoint

Status: active now.

- backend loop works locally
- replay/review summaries work
- two deterministic toy scenarios exist
- environmental cues are represented in backend state
- Unreal text display path has started

Next checkpoint

Target: make Unreal read project data without manual copy/paste.

- read generated debug text file from Unreal
- keep scenario truth in backend
- use this as the bridge before JSON parsing

Next after that

Target: make Unreal display structured state.

- parse display-state JSON in Unreal
- render warnings, errors, and object rows
- still avoid full gameplay logic

First stronger demo checkpoint

Target: live backend-to-Unreal display.

- backend WebSocket sends state
- Unreal receives state
- debug panel updates live
- review summary still explains the session afterward

Later POC checkpoint

Target: make the demo easier to understand visually.

- simple Unreal scene shell
- object placeholders
- reviewed asset workflow through Blender
- no generated assets trusted without review

Future research checkpoints

Target: explore the larger original project vision.

- generated assets
- real physics experiments
- sensing experiments
- AI adaptation
- validation framework expansion
- deployment planning

Rule of thumb

If a feature changes scenario truth, validation, or review, build it in the backend first.

If a feature only displays state or captures input, it can live in a future client layer.

That keeps the system explainable as it grows.